GUIA TÉCNICO

Keras Sequential: regressão e classificação

Capítulo 07/20 · Fundamentos, experiência reproduzível e interpretação

PROPÓSITO E OBJETIVOS

- Montar, compilar e treinar redes densas de forma explícita e econômica.
- Relacionar a técnica à sua função de perda, representação e protocolo de avaliação.
- Explicar limitações e formular um teste reprodutível.

MAPA DE ESTUDO

1. Fundamentação do método
2. Expressões e hipóteses
3. Experimento essencial executável
4. Laboratório aplicado e limitações
5. Exercício e verificações

01 FUNDAMENTAÇÃO CONCEITUAL

Sequential empilha camadas em ordem linear; especificar Input, unidades, ativação, loss adequada e métrica relevante. Redes tabulares devem ser comparadas a baselines clássicos, frequentemente competitivos.

Fonte histórica de apoio: notas do curso original, consultadas apenas como referência de contexto; as explicações e exemplos desta edição são independentes.

Questões críticas: quais pressupostos são necessários? O que a função aprende? Qual métrica indica utilidade real? Que tipo de vazamento ou má interpretação invalidaria a conclusão?

02 MATEMÁTICA E DIMENSÕES

Expressão esquemática: Dense(h): y=f(XW+b)

A expressão acima resume a relação central. Para a derivação, identifique dimensões, hipóteses e operações; não interprete notação compacta como uma implementação completa.

LEITURA CRÍTICA DO CAPÍTULO

A arquitetura Sequential aceita uma sequência linear de camadas, mas deve ser comparada com baseline e avaliada em dados não vistos.

Procedimento: execute a próxima célula com o núcleo declarado em requirements.txt, confira os assert e interprete os resultados. O experimento é independente dos exemplos anteriores.

03 EXPERIMENTO ESSENCIAL

Código extraído do notebook revisado. Execute a célula completa no JupyterLab; as quebras visuais abaixo não substituem o arquivo executável.

```
from sklearn.datasets import make_regression
from sklearn.linear_model import Ridge
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error
X,y=make_regression(n_samples=140,n_features=5,noise=10,random_state=17)
xt,xv,yt,yv=train_test_split(X,y,random_state=17)
base=Ridge().fit(xt,yt)
print('Baseline Ridge MAE:',round(mean_absolute_error(yv,base.predict(xv)),2))
```

INTERPRETAÇÃO

Compare o resultado obtido com a formulação matemática. Confirme shapes, escala e hipótese de modelagem. Um experimento sintético valida uma operação, não substitui avaliação em dados

reais. Ao treinar modelos, separe dados e ajuste transformações apenas no treino; preserve um conjunto final de teste.

04 LABORATÓRIO APLICADO

Laboratório aplicado revisado — MLP real com avaliação contra baseline em dados não vistos

A arquitetura Sequential aceita uma sequência linear de camadas, mas deve ser comparada com baseline e avaliada em dados não vistos.

Procedimento: execute a próxima célula com o núcleo declarado em requirements.txt, confira os assert e interprete os resultados. O experimento é independente dos exemplos anteriores.

```
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPRegressor
from sklearn.dummy import DummyRegressor
from sklearn.metrics import mean_absolute_error
X,y=make_regression(n_samples=220,n_features=5,noise=8,random_state=41)
Xtr,Xte,ytr,yte=train_test_split(X,y,test_size=.25,random_state=41)
base=DummyRegressor().fit(Xtr,ytr)
net=make_pipeline(StandardScaler(),MLPRegressor(hidden_layer_sizes=(24,12),max_iter=550,ra ↪
    ndom_state=41,early_stopping=True,n_iter_no_change=20))
net.fit(Xtr,ytr)
mae_base=mean_absolute_error(yte,base.predict(Xte));mae_net=mean_absolute_error(yte,net.pr ↪
    edict(Xte))
print('MAE baseline:',round(mae_base,2),'MAE MLP:',round(mae_net,2))
assert len(net.predict(Xte))==len(yte)
```

Comparar também com regressão linear: redes profundas não são automaticamente a melhor escolha em dados tabulares.

Checklist: conferir entradas e saídas, sementes, inexistência de uso dos dados de teste durante ajuste e distinção entre uma demonstração sintética e desempenho em aplicações reais.

05 EXERCÍCIO E SOLUÇÃO COMENTADA

Qual loss usar para classificação binária com saída sigmoid?

Resposta: Binary cross-entropy, com rótulos binários e probabilidades coerentes.

LIMITAÇÕES E BOAS PRÁTICAS

- Confundir ajuste no treino com generalização para dados não vistos.
- Reportar somente uma métrica ou uma execução aleatória.
- Ignorar shape, tipo de variável e convenção do backend.
- Executar exemplos com pesos baixados sem registrar versão, licença e procedência.
- Confundir demonstração matemática com validação de um modelo de produção.

VERIFICAÇÃO LOCAL

Execute Reiniciar kernel e executar todas as células. Registre versões do Python e das dependências, confira shapes e métricas e diferencie os laboratórios opcionais dos exemplos NumPy/scikit-learn.

Validação posterior (13/11/2026): notebooks 14 a 20 executados com TensorFlow CPU; opcionais 01 a 13 não integralmente revalidados.

REFERÊNCIAS TÉCNICAS

NumPy: numpy.org/doc · scikit-learn: scikit-learn.org/stable/user_guide.html · Keras: keras.io/guides · TensorFlow: tensorflow.org/guide

REVISÃO DIDÁTICA · CAPÍTULO 07

Sequential: API Keras versus baseline scikit-learn

1. Conceito, escopo e interpretação

Keras Sequential descreve uma cadeia única de camadas. O notebook demonstra modelos scikit-learn (Ridge, DummyRegressor e MLPRegressor) e contém a opção de ambiente TensorFlow/Keras; não trate um MLPRegressor como execução da API Sequential.

2. Formalização e exemplo resolvido

> Exemplo de arquitetura: Input(shape=(p,)) -> Dense(32,relu) -> Dense(1). Em regressão, compile(optimizer=Adam(...),loss=mse); avaliar por MAE no mesmo teste utilizado pelos baselines.

3. Relação com o notebook e experimento

Construa um Sequential opcional com Input e Dense, compile, fit apenas no treino e avalie no teste. Compare o MAE com Ridge, DummyRegressor e MLPRegressor do notebook com mesma partição. Reporte se Keras não estiver disponível.

4. Limitações e erros frequentes

O parâmetro de treinamento, a escala da entrada e o número de épocas influenciam a comparação. Evite declarar que Sequential supera baselines sem evidência experimental.

5. Exercício comentado

Quando Sequential deixa de ser a melhor escolha? Quando o grafo exige múltiplas entradas/saídas, ramificações ou compartilhamento de camadas.

6. Ampliação conceitual do guia

Keras Sequential organiza camadas em sequência linear, com uma entrada principal e saída seguindo um único caminho. O MLPRegressor do scikit-learn é um baseline neural válido, mas não constitui implementação da API Keras Sequential.

7. Orientação de estudo e validação

O notebook possui laboratório aplicado com MLPRegressor, Ridge e DummyRegressor, além de laboratório Keras opcional. Compare modelos somente na mesma divisão treino/teste e na escala adequada. Para implementação Keras, explicite Input, Dense, compile, fit e evaluate, sem tratar ambiente opcional como executado.

8. Exercício adicional de fixação

Exercício: qual API é adequada para dois ramos independentes que depois se fundem? Resposta: Functional; Sequential não representa diretamente esse grafo de múltiplas entradas.

Verificação: execute o notebook homônimo no ambiente apropriado; registre versões, shapes, sementes e métricas. Laboratórios Keras opcionais não foram executados durante esta revisão documental.

Abrir notebook do capítulo no repositório

LABORATÓRIO COMPLEMENTAR · CAPÍTULO 7

Laboratório Keras Sequential — regressão

Implementação ilustrativa / formalização

```
import numpy as np
from keras import Sequential, Input, layers
from keras.optimizers import Adam
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error
rng = np.random.default_rng(42)
X = rng.normal(size=(500, 4)).astype('float32')
y = (2*X[:, 0] - X[:, 1] + .1*rng.normal(size=500))
a,b,ya,yb = train_test_split(X,y,test_size=.2,random_state=42)
model = Sequential([Input(shape=(4,)), layers.Dense(16,activation='relu'),
                    layers.Dense(1)])
model.compile(optimizer=Adam(.005), loss='mse')
model.fit(a, ya, validation_split=.2, epochs=20, verbose=0)
print('MAE teste:', mean_absolute_error(yb,model.predict(b).ravel()))
```

Interpretação e limites

O exemplo demonstra a API Sequential, e não MLPRegressor. Compare o MAE com um DummyRegressor e com Ridge treinados na mesma partição; não interprete o valor impresso como resultado validado neste documento.

Como verificar

Execute o código no ambiente indicado pelo notebook. Se depender de Keras/TensorFlow, registre versões e resultados reais; esta revisão não afirma execução do laboratório.

Referência: notebook do mesmo capítulo em 07_deep_learning (GitHub).